

Saif Elsaady, Yousef El Sheikha

Dr. Berisha

EEE 598 Topic: Speech and Audio Processing and Perception

Due 15 October 2025

Lightweight Keyword Spotting Using Energy and MFCC Features for SoC Deployment

1. **Problem Description**

Keyword Spotting (KWS) is the process of detecting specific spoken words within an audio signal. It is a foundational component of speech-based systems such as voice-activated assistants and command recognition interfaces. Unlike large-scale automatic speech recognition systems that depend on deep neural networks and large datasets, a small-scale keyword spotting system can be implemented using classical speech signal processing techniques that analyze short frames of speech to identify characteristic acoustic patterns.

In this project, a keyword spotting system will be implemented in MATLAB using the fundamental methods learned in this course. The system will use a subset of the Google Speech Commands Dataset, which provides short, one-second recordings of common spoken words from multiple speakers. This allows the system to be trained and tested on real speech data without the need for custom recordings while maintaining manageable data size and complexity.

The system will follow a frame-based processing approach, where each audio signal is divided into short overlapping frames (approximately 20 ms) to capture the time-varying nature of speech. For each frame, time-domain features such as short-time energy and zero-crossing rate (ZCR) will be calculated to represent the intensity and voicing characteristics of the signal. In the frequency domain, the Short-Time Fourier Transform (STFT) will be applied, and the resulting spectrum will be converted to Mel-Frequency Cepstral Coefficients (MFCCs) to represent the perceptually relevant spectral shape of speech.

A lightweight classifier such as k-Nearest Neighbors (k-NN) or Support Vector Machines (SVM) will then be trained to distinguish between a small set of selected keywords (for example,

“yes,” “no”, “stop”, and “go”). The goal is to build a fully functional keyword recognition system using only the fundamental analysis and classification techniques covered in class. The final system will demonstrate how classical signal-processing methods can be combined to achieve effective speech-based word recognition using an existing dataset rather than relying on deep learning frameworks or pretrained models.

Objectives:

- I. Design and implement a complete speech-processing pipeline in MATLAB from scratch, including preprocessing, framing, and feature extraction using short-time energy, zero-crossing rate (ZCR), and Mel-Frequency Cepstral Coefficients (MFCCs).
- II. Load and preprocess a subset of the Google Speech Commands Dataset, selecting several spoken keywords (ex. “yes”, “no”, “stop”, and “go”) along with an additional “unknown/silence” category for classification.
- III. Develop and train a lightweight classifier, such as k-Nearest Neighbors (k-NN) or Support Vector Machines (SVM), to recognize the selected keywords based on extracted features.
- IV. Evaluate the system’s performance using overall classification accuracy and confusion-matrix analysis, and discuss how the choice of features and classifier affects recognition accuracy.

2. Project Approach

Given the concepts and tools covered in this course, this project will be implemented entirely from scratch in MATLAB to build a small-scale keyword-spotting system using the classical speech processing techniques introduced in class. All components, including preprocessing, feature extraction, classification, and performance evaluation, will be manually developed to demonstrate a full understanding of the speech recognition process. The system will analyze short spoken words using frame-based processing, time- and frequency-domain features, and lightweight classification.

To train and evaluate the system, a subset of the Google Speech Commands Dataset (v2) will be used. This dataset provides one-second audio recordings of various spoken keywords from multiple speakers. Using this dataset ensures a consistent and diverse source of real speech samples without the need for new recordings, while still keeping the overall project scope manageable.

Modifications and Enhancements:

A. Signal Preprocessing and Feature Extraction:

Audio samples from the dataset will be resampled to 16 kHz, normalized, and divided into short overlapping frames (approximately 20 ms) using a Hamming window. For each frame, short-time energy and zero-crossing rate (ZCR) will be computed to capture temporal variations and voicing characteristics. In the frequency domain, the Short-Time Fourier Transform (STFT) will be applied to each frame, followed by Mel-scale filtering and a Discrete Cosine Transform to

generate Mel-Frequency Cepstral Coefficients (MFCCs). These features together represent both prosodic and spectral properties of the speech signal and form the input feature vectors for classification. Mel-Frequency Cepstral Coefficients (MFCCs).

- B. **Classifier Development:** A simple machine-learning classifier such as k-Nearest Neighbors (k-NN) or Support Vector Machines (SVM) will be implemented and trained to distinguish between a selected set of keywords (for example, “yes”, “no”, “stop”, and “go”) along with an additional “unknown/silence” category. Different feature combinations (energy w/ ZCR w/ MFCCs vs. MFCCs only) will be compared to evaluate their effect on classification performance.
- C. **Performance Evaluation:** The system’s performance will be evaluated using overall classification accuracy and confusion-matrix analysis to show how well the model correctly identifies each keyword. Additional experiments may include varying the number of MFCC coefficients or testing under moderate background noise to examine robustness. The analysis will highlight how feature selection and classifier complexity affect recognition accuracy, demonstrating the effectiveness of classical speech-processing techniques for real-time keyword recognition.

3. Detailed Time Breakdown

The project is budgeted for approximately 30 hours of effort, allocated across the following stages:

Stage	Activities	Estimated Effort (Hours)
I. Dataset Setup & Preprocessing	Select and download a subset of the Google Speech Commands Dataset containing the target keywords plus “unknown/silence.” Organize data into training, validation, and testing sets. Perform resampling (16 kHz), normalization, and trimming to prepare clean input signals for analysis.	3-4 hours
II. Feature Extraction Pipeline Development	Implement frame-based segmentation (~20 ms windows) and compute short-time energy, zero-crossing rate (ZCR), and Mel-Frequency Cepstral Coefficients (MFCCs). Combine these features into a single feature vector per audio clip and visualize feature behavior.	7 hours
III. Classifier Implementation & Training	Implement and train lightweight classifiers (e.g., k-NN, SVM) to recognize selected keywords. Tune parameters (e.g., number of neighbors, kernel type) and test with different feature combinations (energy + ZCR + MFCCs vs. MFCCs only).	6-7 hours
IV. Testing & Evaluation	Evaluate the model on unseen test data and across multiple speakers. Measure accuracy and generate confusion matrices. Conduct small-scale robustness tests such as adding background noise or varying MFCC coefficients.	4-5 hours
V. Final Analysis & Report Documentation	Summarize key results and observations, create plots of feature distributions and classification performance, and write a 4–5 page final report in IEEE format.	6-7 hours
Total Estimated Effort		~26-30 hours

4. Methods Overview

This keyword spotting system will be implemented entirely from scratch in MATLAB using classical speech signal processing and basic machine-learning techniques. The project will follow a structured 3 stage pipeline consisting of feature extraction, classification, and evaluation.

1. In the feature extraction stage, audio samples from the Google Speech Commands Dataset will first be resampled to 16 kHz and divided into short overlapping frames of approximately 20–25 milliseconds with a 10 millisecond hop size. Each frame will be analyzed in both the time and frequency domains. Short-time energy and zero-crossing rate (ZCR) will be computed to capture temporal and voicing characteristics, while Mel-Frequency Cepstral Coefficients (MFCCs) derived from the Short-Time Fourier Transform (STFT) will represent spectral and perceptual features of speech. These features will be combined to form compact feature vectors representing each spoken keyword.
2. In the classification stage, the extracted feature vectors will be used to train a lightweight supervised classifier, such as k-Nearest Neighbors (k-NN) or a Support Vector Machine (SVM), to recognize a small set of selected keywords (e.g., “yes,” “no,” “stop,” and “go”). The classifier will be evaluated on its ability to correctly identify the spoken word from unseen test samples, and different feature combinations (e.g., MFCCs only vs. MFCCs with energy and ZCR) will be compared to assess their effect on accuracy.
3. Finally, in the evaluation stage, system performance will be measured using classification accuracy and confusion-matrix analysis to assess how effectively the model distinguishes

between keywords. Additional tests with multiple speakers and moderate background noise may be conducted to explore the system's robustness and generalization.

These methods are appropriate because they align directly with the signal-processing and pattern-recognition techniques covered in this course, allowing for a practical demonstration of how frame-based analysis, spectral feature extraction, and classical classifiers can be combined to perform speech recognition using an existing dataset.

5. Data

The project will use a subset of the Google Speech Commands Dataset v2 sourced from kaggle.com database, which contains one-second recordings of spoken words. Four keywords will be selected: “yes”, “no”, “stop”, and “go.” An additional class will represent unknown words or silence. This small vocabulary setup allows for efficient training and evaluation while still providing a practical keyword spotting task.

Example Data Distribution:

Keyword	Training Samples	Validation Samples	Test Samples
Yes	3,800	400	500
No	3,750	390	480
Stop	3,700	380	470
Go	3,800	410	500

Unknown/Silence	4,000	400	500
Total	~19,000	~2,000	~2,500

****Note:** To keep computation manageable, we will sample ~1,000–1,500 clips per target class for training and evaluation.

Table I. Example Data Distribution from the Google Speech Commands Dataset (subset used in this project)

6. Expected Experimental Results

The system is expected to accurately classify short spoken keywords using the features and classifiers developed in MATLAB. Using a subset of the Google Speech Commands Dataset, the overall classification accuracy is anticipated to fall within the 75–90% range, depending on the number of samples, feature combinations, and classifier used. The results will be summarized using confusion matrices, showing how effectively the system distinguishes between individual keywords and the “silence” or “unknown” category.

In addition to accuracy, comparisons will be made between classifiers such as k-Nearest Neighbors (k-NN) and Support Vector Machines (SVM) to determine which method provides the best trade-off between accuracy and computational efficiency. The impact of different feature combinations such as using MFCCs alone versus combining MFCCs with short-time energy or zero-crossing rate (ZCR) will also be analyzed to identify which feature set provides the most consistent performance.

To evaluate robustness, the system will be tested on multiple speakers and under moderate background noise using samples from the dataset to examine its ability to generalize

beyond clean training conditions. The final results will highlight how classical speech-processing techniques can achieve effective keyword recognition accuracy using lightweight, interpretable methods developed from scratch in MATLAB.

7. Use of AI Tools

AI tools were used to assist with brainstorming, text editing, and clarifying questions about the feasibility of our ideas. All the ideas, methods, algorithms, feature designs, and final decisions regarding project scope and content were determined by us. AI assistance was also used to clarify our programming approach ideas for standard MATLAB functions (e.g., frame-based processing and feature extraction setup). During the project itself, AI tools may be used only for general guidance and troubleshooting, such as clarifying MATLAB syntax, understanding built-in function behavior, or checking our logic for non-critical utility code like data loading or plotting.

8. Sources

Kaladin Stormblessed, "Google Speech Commands V2," *Kaggle.com*, 2023.

<https://www.kaggle.com/datasets/sylkaladin/speech-commands-v2>.

Warden, Pete. "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition."

arXiv, 2018, arxiv.org/abs/1804.03209.

Zhang, Yundong, et al. "Hello Edge: Keyword Spotting on Microcontrollers." arXiv, 2017,

arxiv.org/abs/1711.07128.